

Sprint 2D.5 — Rule 1 Equivalence Audit (READ BEFORE DELETING ANYTHING)

Rule 1: Before deleting any inference, verify

Everything inferred today == Already exists in the Execution Graph.

If not, do NOT delete.

Verdict: EQUIVALENCE DOES NOT HOLD — deletion is BLOCKED

The Execution Graph does **not** yet contain everything Script Gen infers today. The legacy inference is **not dead code** — **it is the active fallback** for every scenario the graph does not (yet) own. Deleting it now would break passing tests and drop product coverage. This is exactly the “delete code first” mistake 2D.5 is meant to avoid.

1. How the path is actually selected (the crux)

`src/script-gen/script-gen-engine.ts` (≈ lines 1374–1400) chooses per test case:

```
const graphActions = scenarioNode?.actions;
const { lines } = (graphActions && graphActions.length)
  ? this.emitGraphActionLines(graphActions, ...) //  graph-driven (2D.3)
  : this.tcStepsToCode(steps, ...);              //  LEGACY step-text parser

const graphAssertions = scenarioNode?.assertions;
const assertions = (graphAssertions && graphAssertions.length)
  ? this.emitGraphAssertionLines(graphAssertions, ...) //  graph-driven (2D.4)
  : this.buildTcAssertions(tc.expected_result, ...);   //  LEGACY assertion guessing
```

The KB is explicit that the fallback is intentional and live

(`getScenarioActionTemplate` / `getScenarioAssertionTemplate`):

“returns the authored template when present and `null` otherwise ... On `null`, Script Gen keeps using its legacy step parser / assertion inference, so uncurated scenarios are unaffected.”

So: **no graph template** ⇒ **legacy inference runs**. The switch is presence of `node.actions[]` / `node.assertions[]`, nothing else.

2. What the graph owns TODAY

Layer	Owned by graph	Still legacy
Modules with authored templates	authentication only	opencart, orangehrm, saucedemo (checkout/cart/search), registration, everything else
Scenarios with actions[] + assertions[]	4: auth-pos-valid, auth-neg-wrong-password, auth-neg-empty-fields, auth-neg-unknown-user	all others
Auth scenarios knowingly pending (no template)	—	9, listed in AUTH_SCENARIOS_PENDING_ACTION_TEMPLATE

The 9 pending auth scenarios are blocked on **grammar gaps** — the frozen {action, target, value?, optional?} vocabulary cannot express them:

- auth-neg-locked-user — differentiated only by data role + assertion
- auth-edge-whitespace-case — needs a value **transform** (pad/whitespace/case)
- auth-neg-invalid-identifier-format — needs a value-transform grammar
- auth-sec-injection — needs a payload/value-transform grammar
- auth-edge-password-masking — pure observation, no action sequence
- auth-sec-lockout-threshold — needs **repetition** (repeat failed-login Nx)
- auth-sec-session — session-less protected-resource request (not a form)
- auth-pos-remember-me — needs a **browser restart** across contexts
- auth-pos-logout — **multi-phase** flow (login → logout → re-request)

3. Proof that legacy inference is still EXERCISED by passing tests

tests/unit/script-gen-scenario-fidelity.test.ts (29 tests, currently green) drives the locked-user CSV variants — valid / invalid / whitespace / special / empty / maxlen. Every one of those maps to a **pending / uncured** scenario, so each runs through the **legacy** tcStepsToCode + buildTcAssertions + credential inference (locked_users role).

buildTcAssertions (≈ line 4747) is exactly the text-guessing 2D.5B targets:

```

const lc = exp.toLowerCase();
const isError = /error message|epic sadface|locked out|is required|do not match|
invalid|not match|account is locked/.test(lc);
const isLoginPage = /login page|logged out|cannot access|redirected to the login/.test(
lc);
const isSuccess = /products page|inventory|remains logged in|access all product|
redirected to the products/.test(lc);
const successUrl = this.deriveSuccessUrl(exp);

```

⇒ **Deleting legacy now breaks these 29 tests** (and any non-auth golden), directly failing the DoD line “all existing golden tests pass with byte-identical output.”

4. Script Gen still IMPORTS every module the architecture test must forbid

The proposed permanent architecture test would **fail today** — Script Gen is wired into the inference layer it is supposed to have shed:

Import (in <code>script-gen-engine.ts</code>)	2D.5 target
<code>WorkflowMapper (./workflow-mapper)</code>	2D.5A action inference
<code>AssertionEngine (./assertion-engine)</code>	2D.5B assertion guessing
<code>planVerifications (./verification-standards)</code>	2D.5B assertion planning
<code>ScenarioIntelligence , CredentialResolver (./scenario-intelligence)</code>	2D.5C credential inference
<code>rankLocatorCandidates (../intelligence/element-intelligence)</code>	inference
<code>IntelligenceOrchestrator (../services/intelligence-orchestrator)</code>	inference
<code>in-engine scenario buckets (fileBucketFor... , login/register/cart/search , ≈ 1938–1962)</code>	2D.5D business domains

The 2D.5C credential machinery is real and load-bearing (value transforms live in `src/script-gen/scenario-intelligence/` — `expressions.ts` , `transformers/special-characters.transformer.ts` , etc.) — and it is precisely what the 9 pending scenarios need but the graph grammar cannot yet express.

5. The real prerequisite (what unblocks deletion)

Deletion becomes safe only when the graph owns actions **and** assertions for **every scenario Script Gen is expected to handle** (minimum: everything the goldens + fidelity suite exercise). That requires, in order:

1. **Extend the frozen action grammar** to express the pending patterns (value transforms, repetition, multi-phase flows, browser restart, session-less request) — or consciously **descope** those scenarios out of the product.
2. **Author** action + assertion templates for the remaining auth scenarios and for every other supported module (opencart, orangehrm, saucedemo checkout/cart/search, registration...).
3. Only when graph coverage == 100 % of exercised scenarios, delete the fallback (`tcStepsToCode` , `buildTcAssertions` , credential inference, scenario buckets) and land the architecture test — goldens stay byte-identical **because nothing ever reaches the deleted branch**.

This is a large authoring + grammar effort that tensions against “freeze the infrastructure now.” **That is a strategic decision for the sprint owner** — see the options in the handoff.

Definition-of-Done status (today)

DoD line	Status
Script Gen never parses step text	✗ <code>tcStepsToCode</code> active fallback
Script Gen never guesses actions	✗ legacy path live
Script Gen never guesses assertions	✗ <code>buildTcAssertions</code> regex live
Script Gen never guesses datasets	✗ <code>ScenarioIntelligence</code> credential inference live
Script Gen never identifies scenarios by title	⚠ planner matches titles; buckets in engine
No login/checkout/search business knowledge in Script Gen	✗ file buckets ≈ 1938–1962
All executable behavior comes from the graph	✗ only 4 auth scenarios
Legacy modules deleted, not deprecated	✗ blocked by Rule 1
Goldens byte-identical	✓ today — because legacy still serves them
Script Gen lines decrease	✗ nothing deleted yet

9 of 10 DoD items cannot be satisfied by deletion today. Rule 1 says stop.